BTC Futures Microstructure and Kalshi Contract Price Behavior Dashboard

Site link: BTC Futures & Prediction Contract Behavior · Streamlit
Github repo: SlinkySpace/FA550Final
Video link: https://youtu.be/zye5yQbhcbo
Jordan Ela

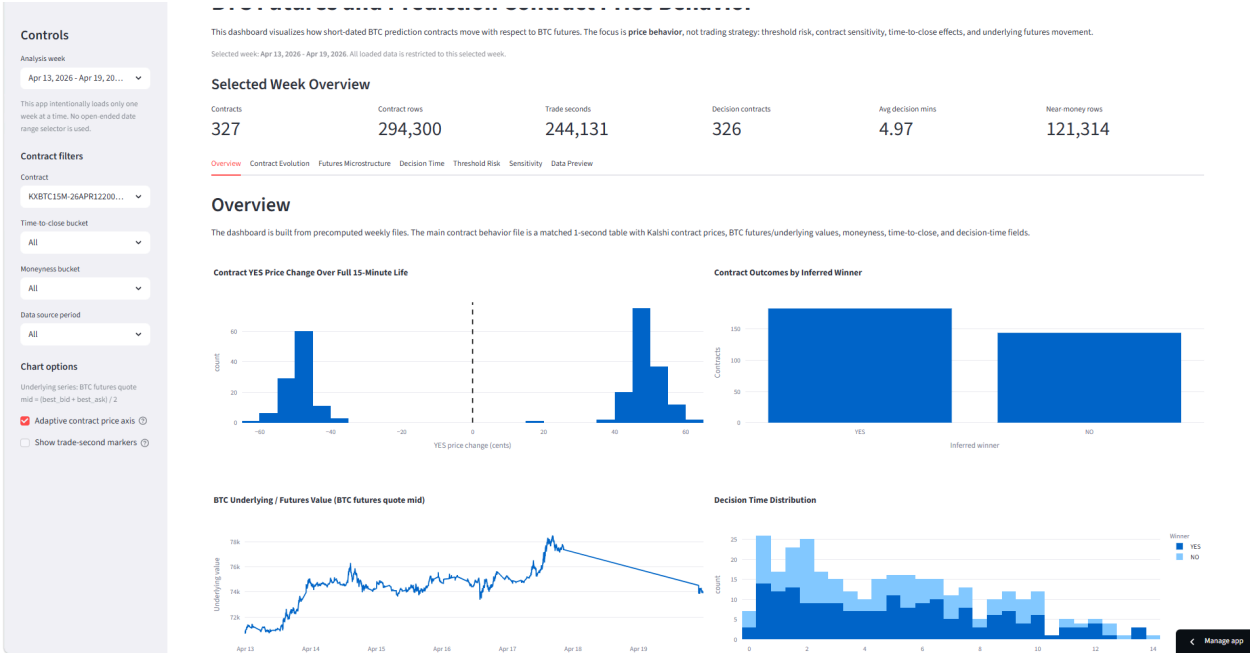
1. Executive Summary

This project is an interactive data visualization dashboard for studying BTC futures microstructure and the price behavior of short-dated Kalshi BTC binary contracts. I built the dashboard to visualize how BTC futures quote movement, market conditions, threshold distance, and time-to-close relate to movement in Kalshi YES/NO contract prices. The project is not a trading strategy and does not recommend trades. Instead, it focuses on visual analysis, market behavior, contract sensitivity, and interpretation.

The dashboard was built with Python, Streamlit, Pandas, Polars, Plotly, and Parquet-based preprocessing. The data came from two main sources. The Kalshi side used Kalshi BTC contract and trade data, including contract-level trade history, YES/NO prices, close times, and contract metadata. The BTC futures side used CME BTC futures market data pulled through Databento, including trades, TBBO quote data, and OHLCV bars. The dashboard uses precomputed weekly files so the app loads only one selected week at a time instead of trying to process the full high-frequency dataset interactively.

The final dashboard includes several connected views: Overview, Contract Evolution, Futures Microstructure, Decision Time, Threshold Risk, Sensitivity, and Data Preview. Together, these views help answer questions such as: When do short-dated binary contracts become effectively decided? How do YES prices respond to short-horizon BTC futures moves? Are contracts more sensitive near the settlement threshold? How do spreads, local volatility, and trading activity

affect contract behavior?



The main findings are that BTC futures microstructure requires careful preparation before visualization, Kalshi binary contracts become especially sensitive near the threshold and near expiration, and weekly preprocessing is necessary to make the dashboard responsive. In the BTC-only microstructure work, the unified 1-second futures table had about 8.55 million rows. After active-market filtering, about 1.78 million rows remained, or roughly 20.8 percent of the original data. This showed that a large share of the raw 1-second data represented inactive or stale periods rather than meaningful market activity.

Key dashboard features include: a weekly selector that prevents the app from loading too much data at once; contract evolution plots comparing Kalshi YES/NO prices with BTC futures quote midpoint; decision-time distributions showing how many minutes before close contracts became effectively decided; threshold-risk and moneyness views showing distance from the settlement boundary; sensitivity plots connecting short-horizon BTC futures moves to short-horizon contract price changes; and a Data Preview tab for auditing the weekly files and derived fields used by the dashboard.

The project is useful for students, instructors, and market-data analysts who want to understand how short-dated binary contracts behave around settlement. Its main value is that it turns a complicated high-frequency dataset into a dashboard that can be explored visually without requiring the user to manually inspect millions of rows.

2. Project Goals and Context

The original motivation for this project was to understand the relationship between BTC futures microstructure and short-dated BTC prediction contracts. BTC futures markets move

continuously and contain detailed quote and trade information, while Kalshi binary contracts convert market movement into yes-or-no event prices. Because these contracts are short-dated, small BTC futures moves near the settlement threshold can cause large changes in contract prices. This makes the relationship visually interesting and financially important.

The main question behind the project is: how do short-dated BTC binary contracts behave relative to BTC futures quote movement, market activity, time-to-close, and distance from the contract threshold?

This matters because binary prediction contracts are becoming a more visible part of financial markets. Unlike regular futures or standard options, a binary contract has a fixed payoff. Once the event is decided, the payout does not keep increasing just because the underlying moves farther past the threshold. This means that binary-contract price behavior is usually most interesting near the decision boundary, where small moves in BTC can cause large changes in implied probability.

My project objectives were to build a working interactive dashboard for BTC futures and Kalshi BTC contract behavior, show how BTC futures-side movement relates to binary contract price movement, visualize threshold distance and decision timing, include microstructure context such as spread and trading activity, and keep the project focused on data visualization rather than trading strategy.

I chose this project because it connects several areas I am interested in: market microstructure, high-frequency financial data, and interactive visualization. I wanted a project that involved messy real data, required meaningful preprocessing, and still led to a dashboard that could explain a financial question visually. BTC futures and Kalshi binary contracts were a good fit because they combine short-horizon price movement, event-based payoffs, and dashboard design challenges.

The intended audience is someone with basic financial market knowledge who wants to understand how a short-dated BTC contract behaves around settlement. This could include a student, instructor, market-data analyst, or someone interested in prediction markets. The dashboard does not assume advanced derivatives knowledge, but it does use terms like futures quote midpoint, spread, threshold, time-to-close, and YES price.

A major scope decision was to avoid presenting this as a trading system. Some earlier side work used related BTC data for trading-strategy experiments, but that is separate from this FA550 visualization assignment. The final project focuses on the visual analytics problem: how to prepare the data, build the dashboard, explain the design choices, and interpret the patterns.

3. Data Preparation Process

The project uses three main categories of data. The first is BTC futures market data from Databento, specifically CME BTC futures trades, TBBO quote data, and OHLCV bars. The second is Kalshi BTC contract and trade data, including contract tickers, YES/NO prices, trade

sizes, open and close times, and metadata. The third is dashboard-ready weekly files generated from preprocessing scripts.

The Databento CME futures data provided the futures-side market context. The TBBO quote data was especially important because the final dashboard uses the futures quote midpoint as the underlying reference:

quote_mid = (best_bid + best_ask) / 2

This quote midpoint gives a consistent futures-side reference path across the weekly dashboard files. The trades and OHLCV files were still useful for checking trade activity, volume, and market movement, but the final contract evolution chart uses quote midpoint because it works consistently across the full dashboard range.

I originally explored adding actual futures trade prices as a selectable underlying source. The issue was that the available actual futures trade file only covered part of the dashboard period. If I kept that option, some weeks would show actual trades and later weeks would not. Since this would create inconsistent behavior across the dashboard, I removed the actual-trade option and used quote midpoint as the single futures-side reference.

The Kalshi side used BTC contract and trade data. These files included YES/NO prices, contract tickers, trade sizes, contract open and close times, and market metadata. After cleaning, the Kalshi data made it possible to build 1-second contract behavior files showing contract evolution, YES/NO price movement, threshold distance, decision timing, and short-horizon sensitivity.

A major data quality issue was that the raw 1-second futures data contained many inactive or stale periods. Many rows had no trades or little meaningful movement. If the full 1-second dataset was used directly, low-volatility regimes mostly captured dead time instead of genuinely calm but active markets. To address this, I used active-market filtering during the BTC-only microstructure exploration. A row was considered active if there was a trade at that second, and active-or-recent if there was a trade within a small window around that second. This made the microstructure analysis more focused on periods where the market was actually moving.

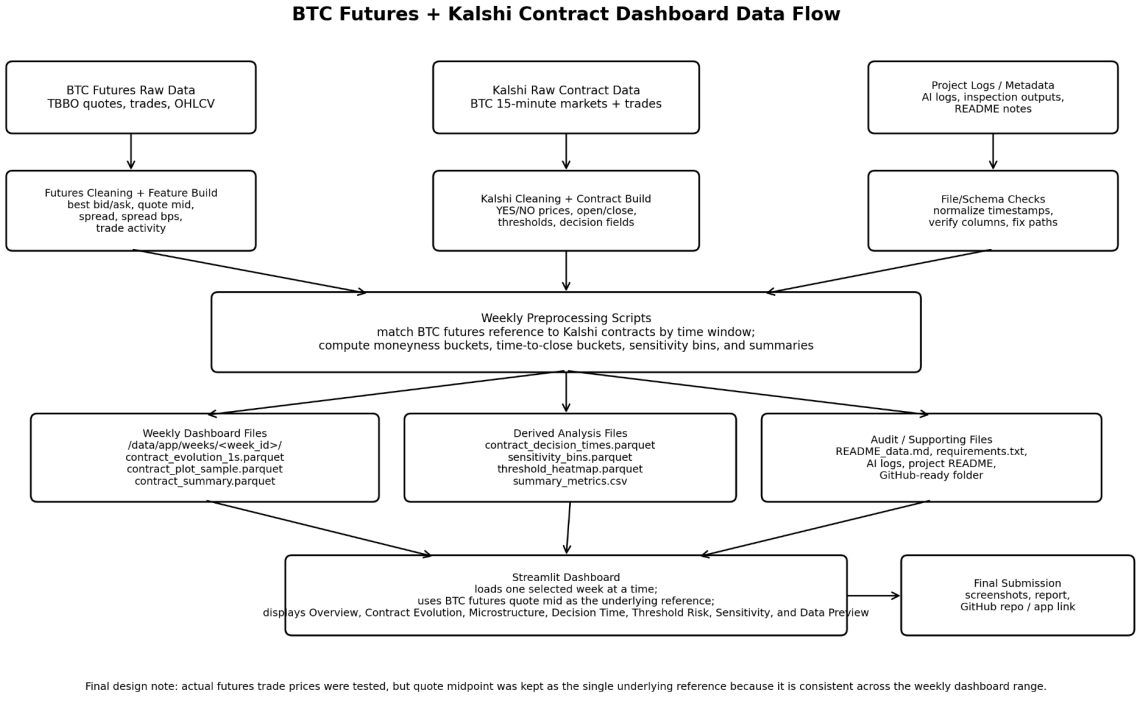
In the earlier BTC-only microstructure analysis, the unified 1-second futures table had about 8.55 million rows. After active filtering, about 1.78 million rows remained, or roughly 20.8 percent of the original data. This was one of the most important data preparation findings because it showed that most raw 1-second rows were not useful for direct event analysis.

The preprocessing pipeline created or used variables such as mid_price, spread, spread_bps, quote_mid, yes_price_cents, no_price_cents, yes_price_change_1s, yes_price_change_5s, underlying_change_1s, underlying_change_5s, z_bps, abs_z_bps, time_to_close_bucket, moneyness_bucket, decision_time, and minutes_before_close_at_decision.

For the final dashboard, the most important preparation decision was to build weekly app files before launching Streamlit. Rather than asking the app to join large futures and Kalshi files interactively, preprocessing scripts generated weekly files in this structure:

data/app/weeks/<week_id>/

Each weekly folder includes files such as contract_summary.parquet, contract_evolution_1s.parquet, contract_plot_sample.parquet, sensitivity_bins.parquet, threshold_heatmap.parquet, contract_decision_times.parquet, and summary metric CSV files. This design made Streamlit function as a visualization layer rather than as a heavy data-processing engine.



Final design note: actual futures trade prices were tested, but quote midpoint was kept as the single underlying reference because it is consistent across the weekly dashboard range.

There were also schema and path issues during setup. One major debugging issue came from a Polars join where both datasets had a window_start column, but the timestamp precision differed. One side used datetime microseconds with UTC, while the other used datetime nanoseconds with UTC. The solution was to normalize timestamp columns before joining. Another setup issue came from file paths. Earlier scripts referred to absolute Desktop or OneDrive locations, which would make the project harder to submit. The final GitHub-ready structure uses relative project paths inside the final project folder.

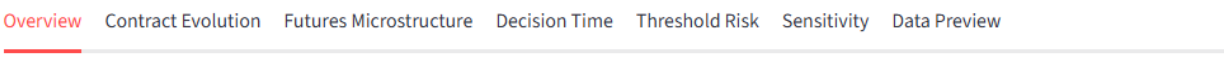
4. Design Decisions and Rationale

The main design goal was to make a complicated market dataset understandable without overwhelming the user. The dashboard needed to connect BTC futures movement, Kalshi

contract prices, threshold distance, and time-to-close. Because the data is high-frequency and large, the dashboard also needed to avoid plotting too many raw points at once.

The first major design choice was the weekly selector. A normal open-ended date range selector seemed attractive at first, but it made the app too easy to overload. If a user selected too much data, the dashboard could freeze or become very slow. The weekly selector solved this problem by forcing the app to load one prepared weekly folder at a time. This made the interaction model simpler and more stable.

The second major design choice was the tab structure. The final dashboard is organized into Overview, Contract Evolution, Futures Microstructure, Decision Time, Threshold Risk, Sensitivity, and Data Preview. The Overview page introduces the selected week and summary metrics. The Contract Evolution page shows how one contract moves. The Microstructure page explains the futures-side environment. The Decision Time page focuses on when contracts become effectively decided. The Threshold Risk and Sensitivity pages explain the relationship between futures movement and contract repricing. The Data Preview page supports auditability.

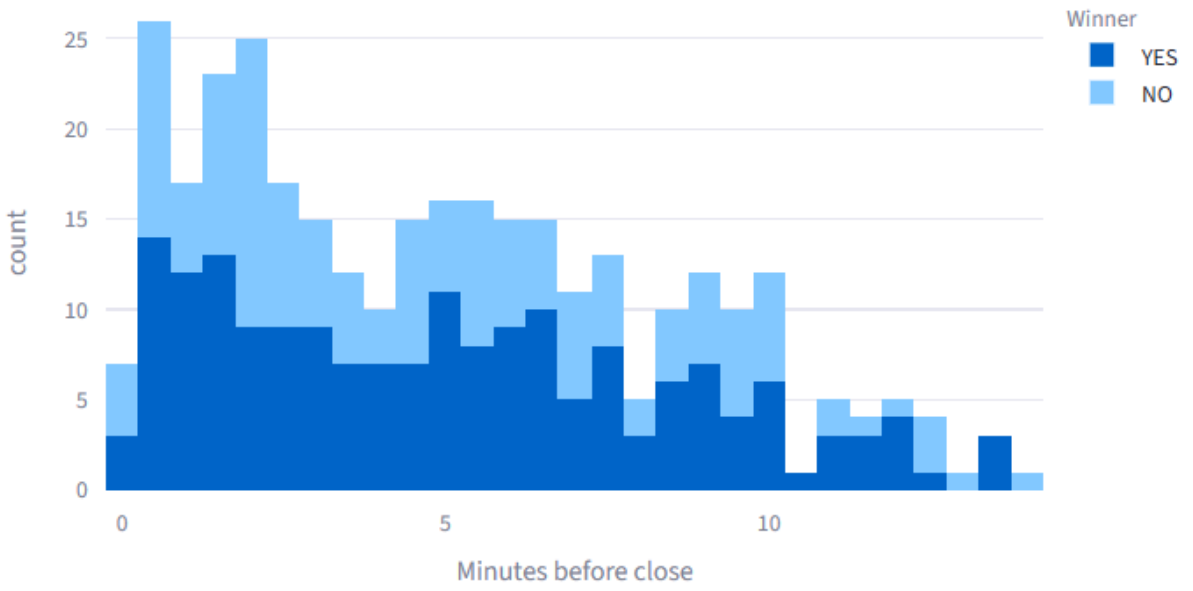


The Contract Evolution page is one of the most important parts of the project. It compares Kalshi YES/NO contract prices with the BTC futures quote midpoint. The quote midpoint is calculated from best bid and best ask, which gives a consistent futures-side reference path. I considered showing actual futures trade prices, but the available actual-trade data did not cover the full dashboard period. I also tested a separate underlying_value option, but it produced the same visual path as quote midpoint. To avoid a confusing control that did not add useful information, the final dashboard uses quote midpoint only.

A duplicate contract-only line chart was removed because it repeated information already shown in the combined contract/underlying chart. It was replaced with distributions of 5-second YES price changes and box plots of absolute 5-second YES moves by time-to-close bucket. This improved the page because it showed not only where the contract price went, but also how jumpy or stable it was over short intervals.

The Average Paths page was also removed. The original idea was to show average contract paths by groups like time-to-close bucket and moneyness bucket. However, these buckets change over a contract's life. Grouping paths by dynamic fields created disconnected slivers instead of meaningful smooth paths. Removing this tab improved clarity because the final dashboard only keeps charts that answer a clear question.

The Decision Time page visualizes when contracts become effectively decided. The important metric is minutes before close at decision time. This means the chart measures how many minutes before settlement a contract became effectively decided, not how many minutes after open the decision happened. This distinction matters because short-dated binary contracts are strongly shaped by the approach to expiration.

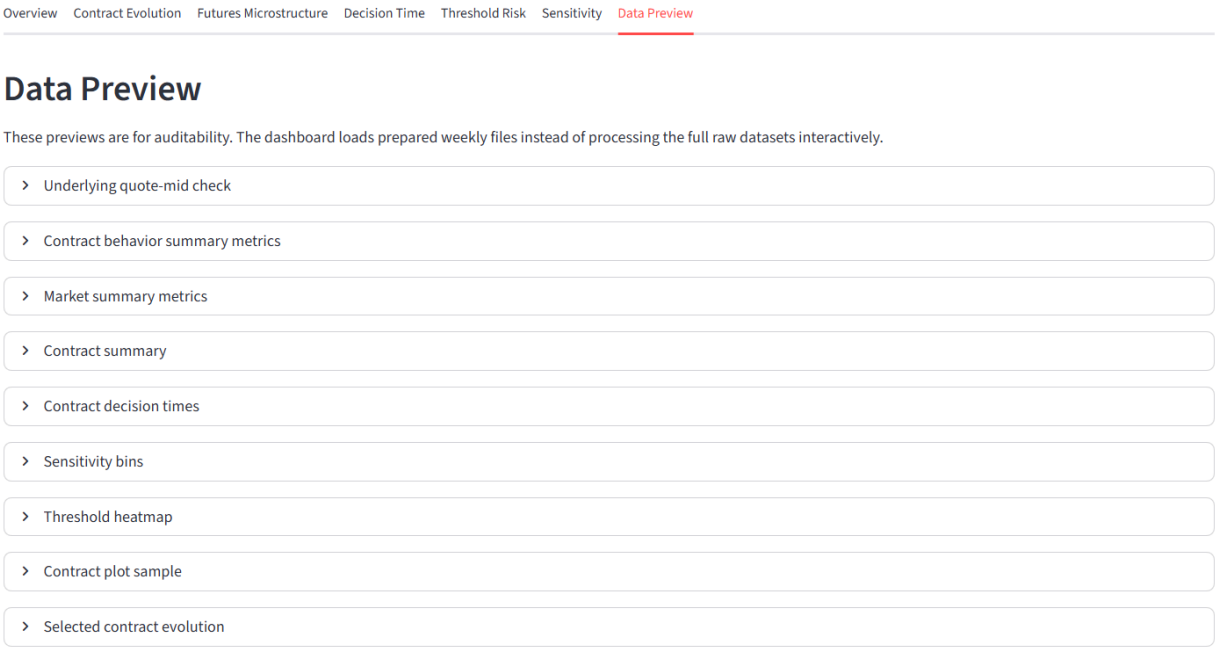


The Threshold Risk page uses the idea of moneyness, but adapted to binary contracts. For a regular call option, being farther above the strike increases intrinsic value. For a binary contract, payoff is fixed. Once the contract settles on the winning side, the payout does not keep increasing with BTC price. Therefore, moneyness is better understood as distance from the settlement boundary. The closer the futures-side reference price is to the threshold, the more sensitive the contract may be to small BTC moves.

Most charts use a simple blue-heavy Plotly style because it keeps the dashboard visually consistent and easy to read. Color is mainly reserved for categorical comparisons such as YES vs NO outcomes, inferred winner groups, and time-to-close buckets. Contract price charts use cents on a 0–100 scale when appropriate because Kalshi contracts settle between 0 and 100 cents. For selected-contract charts, I also used adaptive axes where helpful so smaller price changes were not visually flattened.

The Data Preview page is a deliberate design feature, not just a debugging leftover. Since the project combines multiple processed files and derived metrics, the user needs a way to verify what is actually being loaded. This page shows contract summary data, decision-time data, sensitivity bins, threshold heatmaps, selected contract evolution, and underlying quote-mid

checks. It improves transparency and makes the project easier to explain.



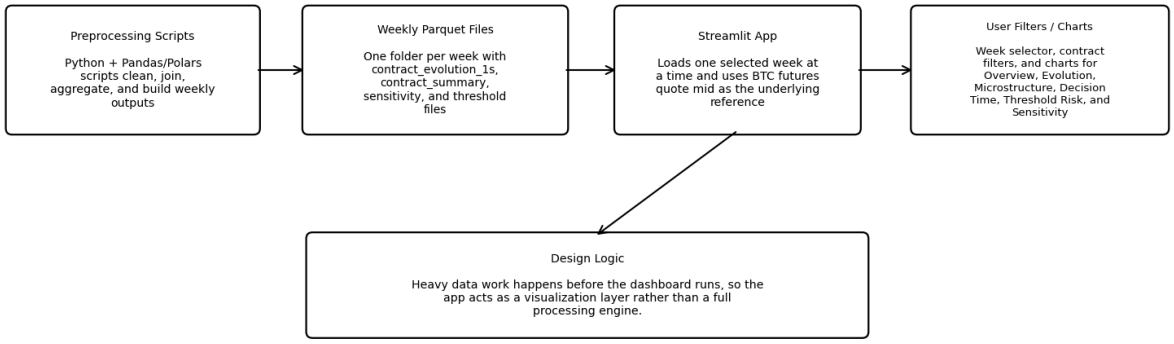
5. Technical Implementation

The project was implemented mainly in Python. Streamlit was used for the interactive dashboard, Plotly for charts, Pandas and Polars for data handling, and Parquet for efficient storage of preprocessed files. The development workflow used VS Code, PowerShell, Git, and GitHub. The requirements.txt file lists the Python packages needed to run the app, including Streamlit, Pandas, Polars, Plotly, NumPy, and PyArrow.

The final technical design separates preprocessing from visualization. The preprocessing scripts perform heavy work such as cleaning data, joining futures and contract files, normalizing timestamps, computing contract-level summaries, creating sensitivity bins, and writing weekly

Parquet files. The Streamlit app then loads the selected week and displays the prepared files.

Architecture Diagram: Preprocessing Scripts -> Weekly Parquet Files -> Streamlit App -> User Filters / Charts



Final design note: quote midpoint is used as the single consistent futures-side reference across the dashboard.

The app uses relative paths so it can run from the project folder without relying on a specific Windows Desktop or OneDrive location. A simplified version of the path logic is:

```
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[1]
DATA_DIR = PROJECT_ROOT / "data"
WEEK_DIR = DATA_DIR / "app" / "weeks"
```

This matters because earlier versions had path problems where scripts looked outside the project folder. By basing paths on the current project root, the dashboard became easier to submit and easier to run on another machine.

A second important implementation feature is weekly loading. The app uses a week index file and weekly data folders, which avoids loading every contract and every second of data at once. A simplified version of the idea is:

```
week_index = pd.read_csv("data/app/week_index.csv")
selected_week = st.sidebar.selectbox("Analysis week", week_index["display"])
selected_week_dir = WEEK_DIR / week_id
contract_summary = pd.read_parquet(selected_week_dir / "contract_summary.parquet")
contract_evolution = pd.read_parquet(selected_week_dir / "contract_evolution_1s.parquet")
```

Another technical issue involved Plotly vertical lines on datetime axes. The app originally used Plotly’s add_vline with annotations to mark decision and close times. This caused a datetime-related error involving Timestamp arithmetic. The solution was to use Plotly shapes

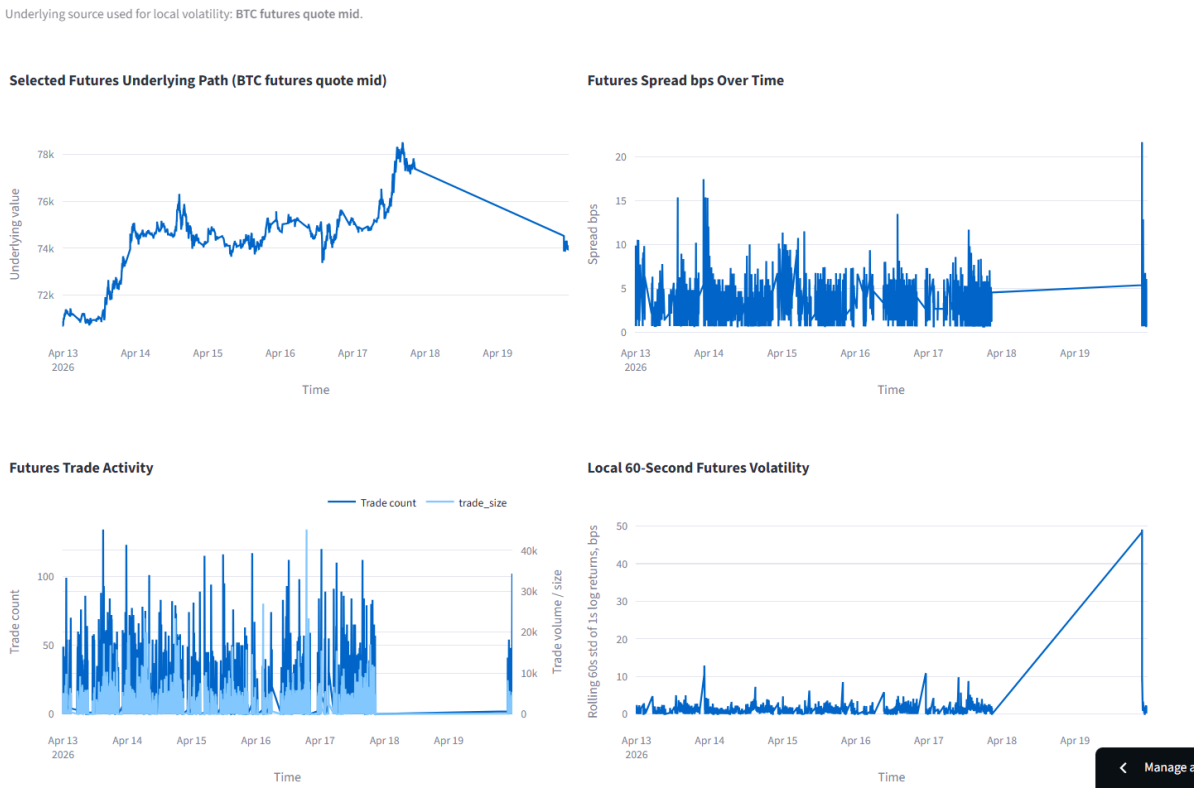
and annotations instead of add_vline. This avoided the crash and kept the decision and close markers on the chart.

A later technical issue involved chart rendering. Some charts showed a WebGL browser warning after the app was revised. The solution was to force SVG rendering for relevant Plotly line and scatter charts. This made the dashboard more reliable in the browser environment used for testing.

The final app uses one underlying reference: BTC futures quote midpoint calculated from best bid and best ask. Earlier versions included multiple underlying-source options, but testing showed that this added confusion without improving the final dashboard. The quote midpoint was the most consistent choice across the weekly dashboard range.

6. Key Findings and Insights

The first major finding is that BTC futures microstructure requires careful filtering. The raw 1-second futures dataset contains many inactive or stale periods. If these periods are included without adjustment, they can dominate the analysis and make low-volatility periods look more meaningful than they really are. In the BTC-only microstructure work, the active-market filter reduced the dataset from about 8.55 million rows to about 1.78 million rows, retaining roughly 20.8 percent of the original data. This showed that filtering was not just a performance step; it changed the interpretation of the data.

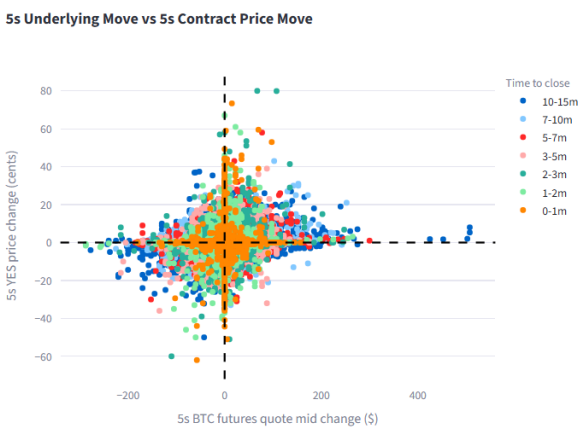


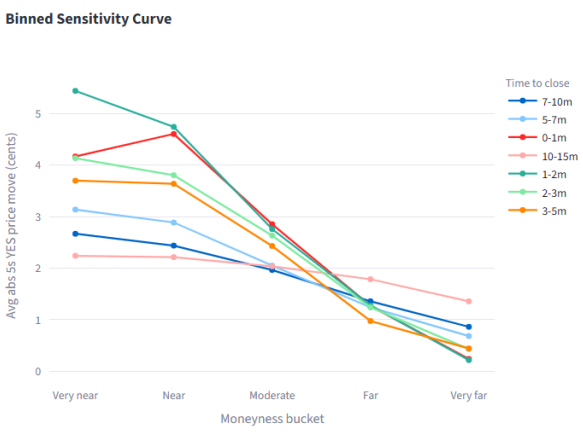
The second finding is that quote midpoint is a useful dashboard reference because it is available consistently across the weekly app files. Actual futures trade prices are useful conceptually, but in the available local data they did not cover the full dashboard period. Since the goal was to build a consistent and reliable dashboard, quote midpoint was the better final reference.

The third finding is that Kalshi YES/NO contract prices often move sharply as the contract approaches expiration. The 5-second YES price-change distributions and time-to-close box plots show that contract repricing can be concentrated in short periods. This supports the idea that binary contracts behave differently from ordinary linear futures exposure.

The fourth finding is that binary-contract moneyness is best interpreted as distance from the decision boundary. This is different from a regular option, where being farther in the money means more intrinsic value. In a binary contract, the payout is fixed. A contract does not become more valuable at settlement just because BTC finishes far above the threshold instead of slightly above it. What matters visually is whether BTC is close enough to the threshold that small futures moves can change the implied probability of the event.

The fifth finding is that time-to-close changes the meaning of price movement. A small BTC move far from expiration may not have the same contract-price impact as the same BTC move near settlement. Near expiration, there is less time for the market to reverse, so binary contract prices can become more sensitive to small futures movements. This is why the dashboard includes time-to-close buckets, decision-time distributions, and short-horizon sensitivity charts.





The sixth finding is that some visual ideas sound good in theory but fail in practice. The removed Average Paths tab is a good example. Averaging contract paths by dynamic buckets created disconnected and misleading visuals. This was an important design lesson. A polished dashboard should not keep a chart just because it was planned. If a chart does not clearly answer a question, it should be revised or removed.

A major limitation is that the final dashboard uses quote midpoint rather than actual futures trade prices as the underlying reference. I tested actual futures trade prices, but the available

live trade file only covered part of the full dashboard range, so using it would have created inconsistent behavior across weeks. Another limitation is that threshold distance is based on derived proximity fields, so future work should recompute it directly from the cleanest threshold and quote-mid fields. Finally, the dashboard is descriptive rather than causal. It shows relationships between BTC futures movement and Kalshi contract behavior, but it does not prove that one variable caused another.

Overall, the project shows that BTC futures and Kalshi binary contracts are connected, but the relationship is not one simple line chart. The connection depends on threshold distance, time-to-close, market activity, data quality, and how the futures-side reference price is constructed. The dashboard's value is that it puts these pieces together in one interactive environment.

7. Challenges and Solutions

One major challenge was performance. The BTC and Kalshi data are large, and early dashboard versions tried to plot too many points directly. This caused freezing and slow interaction. The solution was to precompute weekly dashboard files and make the app load only one week at a time. This turned Streamlit into a visualization layer rather than a full processing engine.

A second challenge was inactive market data. The full 1-second futures dataset contained many rows with no trades or stale quotes. This distorted volatility and event analysis. The solution was to create an active-market filter during the microstructure exploration based on whether trades happened at or near a given second. This made the analysis more focused on real market activity.

A third challenge was path portability. Earlier scripts used absolute paths tied to a specific Desktop or OneDrive folder. That made the project harder to move into a clean GitHub-ready folder. The solution was to organize a final project directory with app files, scripts, data, documentation, and AI logs, and to use relative paths in the app.

A fourth challenge was schema mismatch during weekly preprocessing. A Polars join failed because two window_start columns had different datetime precisions. The solution was to normalize timestamp columns before joining. This fixed the build process and allowed weekly dashboard files to be generated consistently.

A fifth challenge was visualization quality. Some planned charts were redundant or misleading. The Average Paths chart did not work well because grouping by changing fields broke the path into disconnected pieces. A duplicate price chart also did not add much value. The solution was to remove or replace weak visuals with clearer ones, such as 5-second YES price-change distributions and absolute-move box plots.

A sixth challenge was underlying-source selection. I explored using actual futures trades, quote midpoint, and the precomputed underlying_value field. Actual futures trades were only available

for part of the date range, and the underlying_value option produced the same visual path as quote midpoint. The solution was to remove the confusing source selector and use quote midpoint as the single futures-side reference.

A final challenge was browser rendering. One dashboard version triggered a WebGL support warning in the browser. The solution was to force SVG rendering for the relevant Plotly charts, which made the dashboard more stable for presentation and screenshot purposes.

8. AI Usage Throughout the Project

AI tools were used throughout the project, mainly as a coding, debugging, design, and interpretation assistant. AI was not used to replace the project work. Instead, it helped organize the workflow, diagnose errors, explain confusing metrics, and revise the dashboard design.

AI was most helpful in five areas. First, it helped with project scoping. The dashboard originally risked becoming too broad by mixing BTC microstructure, Kalshi contract behavior, Polymarket comparisons, and trading strategy ideas. The final direction narrowed the project to BTC futures microstructure and Kalshi contract price behavior. The trading-strategy component was intentionally excluded because it was separate from the FA550 visualization assignment, even though it used some related underlying data.

Second, AI helped with data setup and file organization. It helped revise scripts to use relative paths, match the VS Code terminal workflow, organize project files inside a GitHub-ready folder, create README files, and identify which scripts and weekly files needed to be included.

Third, AI helped debug technical errors. Examples included a Polars timestamp precision mismatch, missing week index files, a missing week_label column, browser WebGL rendering problems, and confusion between quote midpoint, precomputed underlying_value, and actual futures trade prices.

Fourth, AI helped improve visualization design. It suggested a dashboard structure with Overview, Contract Evolution, Futures Microstructure, Decision Time, Threshold Risk, Sensitivity, and Data Preview pages. It also helped remove weak or redundant controls when they did not communicate clearly.

Fifth, AI helped clarify financial interpretation. For example, it explained that decision time meant minutes before close at decision, not minutes since open. It also clarified that binary-contract moneyness measures distance from the threshold rather than increasing intrinsic payoff.

Example prompts included:
“Why does this moneyness plot look asymptotic near close?”
“Revise the dashboard so it loads only one week at a time and does not crash.”
“Why does quote mid look smoother than actual trades?”
“Remove the actual trades option and use quote mid only.”

AI struggled when the problem required direct inspection of local data files it could not access. In those cases, I had to run scripts locally, paste outputs, and decide whether the result made sense. AI also sometimes suggested visuals or source options that sounded reasonable but did not work well once tested against the actual data. The actual-trade option is an example of this: it worked only for part of the data range, so it was removed from the final dashboard.

The main lesson from using AI was that it works best as an iterative assistant. It was useful for debugging, structure, and explanation, but the final decisions still depended on testing the app, inspecting outputs, and checking whether the visualizations made sense.

9. User Testing and Feedback

Formal user testing was limited, but the project went through several rounds of informal testing during development. The most important testing came from running the dashboard locally, interacting with filters, selecting contracts, and checking whether the charts loaded quickly and made sense.

The main testing task was to select a week, choose a contract, move through each tab, and check whether the charts loaded quickly and matched the selected filters. I also tested edge cases such as later weeks where actual futures trade prices were unavailable, weeks with large samples, and browser rendering issues.

The first major feedback was performance-related. When the dashboard tried to load or visualize too much data, it became slow or froze. This led to the weekly-selector design and precomputed weekly files.

The second major feedback was visual clarity. Some charts looked impressive but did not answer a clear question. The Average Paths chart was removed after it produced disconnected visual fragments. A redundant contract-only line chart was replaced with contract price-move distributions.

The third major feedback was interpretability. Labels such as “average decision minutes” needed to be clearer because they could be misunderstood. The final wording describes this as average minutes before close or average decision lead time.

The fourth major feedback was auditability. Because the dashboard uses multiple data files and derived fields, the Data Preview tab was kept. This allows the user to inspect contract summaries, evolution data, sensitivity bins, threshold heatmaps, and selected contract evolution.

The fifth major feedback was control simplification. The dashboard originally had an underlying-source dropdown, but testing showed that the model/underlying_value option and quote-mid option produced the same chart, while actual trade prices did not cover the full data range. Removing the dropdown made the final dashboard easier to understand and reduced the chance of misleading interpretation.

10. Reflection and Future Directions

This project changed significantly from the original broad idea. At first, the project was more open-ended and included BTC futures, Kalshi, Polymarket, possible trading implications, and microstructure analysis. Over time, the project became more focused. The final version is best described as a dashboard for BTC futures microstructure and Kalshi BTC binary contract price behavior.

The biggest thing I learned is that data visualization is not just about making charts. It is also about choosing what not to show. Some planned visuals were removed because they were confusing, redundant, or misleading. This made the final dashboard stronger. I also learned that interactivity can be a problem if it is not constrained. A dashboard with unlimited date selection may sound better, but in this case a weekly selector created a better user experience.

Another important lesson was that high-frequency market data requires careful preparation before visualization. Raw 1-second data can contain many stale rows, inactive periods, and odd observations. If these are not handled, the dashboard may technically work but tell the wrong story. Active filtering, weekly preprocessing, and data preview checks helped make the analysis more reliable.

The project also improved my understanding of binary contracts. Unlike standard options, binary contracts have fixed payoff. This changes how moneyness and threshold distance should be interpreted. The most important question is not how far the contract is in the money in a standard payoff sense, but how close the underlying futures-side reference price is to the decision boundary and how much time remains before settlement.

If I had more time, I would improve the project in several ways. First, I would recompute threshold distance directly from the cleanest available threshold and quote-mid fields, instead of relying only on precomputed proximity columns. Second, I would add a stronger validation section comparing quote midpoint, OHLCV close, and actual trade prices where all three are available. Third, I would add more contract-level summaries that compare behavior across many weeks. Fourth, I would improve the user guide so someone unfamiliar with the project could quickly understand how to use each tab.

A future version could also compare Kalshi and Polymarket BTC contracts more directly. However, that would require careful alignment of contract definitions, thresholds, settlement rules, and timestamps. For this submission, keeping the project focused on BTC futures and Kalshi BTC contract behavior was the better choice.

Overall, the project produced a working visualization tool that connects BTC futures microstructure with short-dated binary contract behavior. It shows how futures quote movement, threshold distance, decision timing, and market activity interact. The final result is not a trading system. It is a visual analytics dashboard for understanding how these markets behave.

References and Supporting Materials

The project uses locally processed BTC futures data from Databento CME futures files, Kalshi BTC contract and trade data, weekly dashboard files, project scripts, and AI conversation logs included in the final submission package.

Supporting materials include:

Final Streamlit app files.
Weekly dashboard data files.
Data README explaining each dataset.
Project README with setup instructions.
Requirements file.
AI logs covering data setup, weekly build debugging, dashboard visualization design, moneyness interpretation, and key project decisions.
User guide or quick-start instructions.

I pledge on my honor that I have not given or received any unauthorized assistance on this assignment/examination. I further pledge that I have not copied any material from a book, article, the Internet or any other source except where I have expressly cited the source. I have documented all AI tool usage including prompts and tool selection rationale.

Signature: Jordan Ela Date: 5/10/26